Adatbázis és PL/SQL Teljes Fogalomtár – ZH2 Felkészítő

Ez a tudástár a `zh2_felkeszito.md`, a `proba_zh.pdf` és az órai gyakorlati anyagok alapján rendszerezi az Adatbázisok 2 második zárthelyi dolgozatához szükséges összes adatbázis- és PL/SQL fogalmat. Minden fogalomhoz tartozik egy tömör definíció, a leggyakoribb szintaxis és egy gyakorlati példakód.

---

1. Architektúra és Blokkszerkezet

1.1 Névtelen blokk (Anonymous Block)

- **Definíció:** Olyan lefordítás nélkül azonnal futtatott PL/SQL kódblokk, amely nem tárolódik az adatbázis katalógusában. Három fő szekcióból áll: deklarációs (`DECLARE`), végrehajtható (`BEGIN`) és kivételkezelő (`EXCEPTION`).
- **Példa:** `sql DECLARE v_szam NUMBER := 10; BEGIN DBMS_OUTPUT.PUT_LINE('Érték: ' || v_szam); EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Hiba lépett fel!'); END; /`

1.2 Utasítás-elválasztó perjel (`/`)

- **Definíció:** Az Oracle SQL*Plus és PL/SQL Developer eszközök számára jelzi a bufferben lévő PL/SQL blokk vagy DDL utasítás lezárását és végrehajtását.
- **Szabály:** Mindig új sor elején, önmagában álljon a blokk lezáró `END;` sora után.

---

2. Változók és Attribútumtípusok

2.1 Skalár típusok és Konstansok

- **Definíció:** Egyetlen diszkrét értéket tároló adattípusok (`VARCHAR2`, `NUMBER`, `DATE`, `BOOLEAN`). A `CONSTANT` kulcsszóval deklarált változók értéke nem módosítható a blokk futása során.
- **Példa:** `sql v_szoveg VARCHAR2(100) := 'Kezdőérték'; v_ar NUMBER(10,2) := 1500.50; v_datum DATE := SYSDATE; b_aktiv BOOLEAN := TRUE; -- csak PL/SQL-ben létezik, SQL táblában nem! c_ado CONSTANT NUMBER := 0.27;`

2.2 `%TYPE` – Oszlop típusának átvétele

- **Definíció:** Dinamikus típus-hozzárendelés. A változó típusát és hosszát közvetlenül egy létező táblaoszlopból (vagy másik változóból) örökli, így sémaváltozás esetén a kód automatikusan adaptálódik.
- **Példa:** `sql v_megnevezes termek.megnevezes%TYPE; v_ar bolt_termek.egysegar%TYPE;`

2.3 `%ROWTYPE` – Teljes sor típusának átvétele

- **Definíció:** Olyan rekordtípus, amely egy adatbázistábla, nézet vagy explicit kurzor teljes oszlopszerkezetét egyetlen összetett változóként képezi le.
- **Példa:** `sql DECLARE v_bolt bolt%ROWTYPE; BEGIN SELECT * INTO v_bolt FROM bolt WHERE id = 12300; DBMS_OUTPUT.PUT_LINE(v_bolt.nev || ' (' || v_bolt.cim || ')'); END; /`

---

3. Vezérlési Szerkezetek és Feltételes Elágazások

3.1 `IF-THEN-ELSIF-ELSE`

- **Definíció:** Feltételes elágazáskezelés. Ha az `IF` vagy `ELSIF` feltétel igazra (`TRUE`) értékelődik, a hozzá tartozó utasítások futnak le; egyéb esetben az `ELSE` ág fut le.
- **Példa:** `sql IF v_count = 0 THEN RETURN 'Nincs termek!'; ELSIF v_count = 1 THEN RETURN v_egy_termek; ELSE RETURN v_lista; END IF;`

3.2 Simple CASE és Searched CASE

- **Definíció:**
- **Simple CASE:** Egyetlen kifejezés pontos egyenlőségét vizsgálja adott értékekkel szemben.
- **Searched CASE:** Tetszőleges független logikai feltételeket értékel ki fentről lefelé.
- **Példa (Searched CASE):** `sql CASE WHEN v_keszlet = 0 THEN DBMS_OUTPUT.PUT_LINE('ELFOGYOTT'); WHEN v_keszlet < 50 THEN DBMS_OUTPUT.PUT_LINE('UTÁNTÖLTÉS SZÜKSÉGES'); ELSE DBMS_OUTPUT.PUT_LINE('ELEGENDŐ KÉSZLET'); END CASE;`

---

4. Ciklusok (Loops)

4.1 Alap LOOP (`EXIT WHEN`)

- **Definíció:** Végtelen ciklus, amelyből a feltételhez kötött `EXIT WHEN` utasítással lépünk ki.
- **Példa:** `sql LOOP v_idx := v_idx + 1; EXIT WHEN v_idx > 5; END LOOP;`

4.2 WHILE LOOP

- **Definíció:** Elöltesztelő ciklus; addig ismétlődik, amíg a fejlécben szereplő feltétel `TRUE` .
- **Példa:** `sql WHILE v_idx IS NOT NULL LOOP DBMS_OUTPUT.PUT_LINE(v_idx || ' -> ' || v_map(v_idx)); v_idx := v_map.NEXT(v_idx); END LOOP;`

4.3 FOR LOOP és REVERSE FOR LOOP

- **Definíció:** Számlálós ciklus előre meghatározott intervallumra (`alsó_érték .. felső_érték`). A ciklusváltozót a PL/SQL automatikusan deklarálja. A `REVERSE` kulcsszóval felülről lefelé számol.

- **Példa:** ```sql FOR i IN 1..5 LOOP DBMS_OUTPUT.PUT_LINE('Növekvő: ' || i); END LOOP;

FOR i IN REVERSE 1..5 LOOP DBMS_OUTPUT.PUT_LINE('Csökkenő: ' || i); END LOOP; ```

---

5. Kurzorok és Attribútumaik

5.1 Implicit kurzor (`SELECT ... INTO`)

- **Definíció:** Olyan beágyazott SQL lekérdezés, amelynek eredményét közvetlenül PL/SQL változókba töltjük be.
- **Szabály:** Pontosan **egy** sort kell visszaadnia. Ha 0 sort ad → `NO_DATA_FOUND`, ha 2 vagy több sort → `TOO_MANY_ROWS` kivétel keletkezik.
- **Példa:** `sql SELECT megnevezes INTO v_nev FROM termek WHERE id = 9000;`

5.2 Explicit kurzor (`OPEN` , `FETCH` , `CLOSE`)

- **Definíció:** Névvel ellátott SQL lekérdezés, amelynek sorait a fejlesztő explicit lépésekkel nyitja meg, olvassa be és zárja le.
- **Példa:** `sql DECLARE CURSOR cur_boltok IS SELECT nev, cim FROM bolt ORDER BY nev; v_rec cur_boltok%ROWTYPE; BEGIN OPEN cur_boltok; LOOP FETCH cur_boltok INTO v_rec; EXIT WHEN cur_boltok%NOTFOUND; DBMS_OUTPUT.PUT_LINE(v_rec.nev); END LOOP; CLOSE cur_boltok; END; /`

5.3 Kurzor FOR LOOP (deklarált és inline)

- **Definíció:** A legbiztonságosabb és legtisztább kurzorbejárási forma. Az Oracle automatikusan megnyitja, soronként kiolvassa és a végén lezárja a kurzort.
- **Példa (Inline kurzor):** `sql FOR rec IN (SELECT nev, cim FROM bolt ORDER BY nev) LOOP DBMS_OUTPUT.PUT_LINE(rec.nev || ' - ' || rec.cim); END LOOP;`

5.4 Paraméteres kurzor

- **Definíció:** Olyan kurzor, amely bemeneti argumentumokat fogad a `WHERE` feltétel dinamikus szűréséhez.
- **Példa:** `sql CURSOR cur_termekek(p_bolt_id NUMBER) IS SELECT t.megnevezes, bt.egysegar FROM bolt_termek bt JOIN termek t ON t.id = bt.termek_id WHERE bt.bolt_id = p_bolt_id;`

5.5 Kurzor attribútumok

- **Explicit kurzorokhoz (`cur%`):**
- `%FOUND` : Az utolsó `FETCH` sikeresen olvasott-e be sort (`BOOLEAN`).
- `%NOTFOUND` : Az utolsó `FETCH` nem talált több sort (`BOOLEAN`).
- `%ROWCOUNT` : Az eddig beolvasott sorok darabszáma (`NUMBER`).
- `%ISOPEN` : A kurzor nyitva van-e éppen (`BOOLEAN`).
- **Implicit SQL műveletekhez (`SQL%`):**
- `SQL%FOUND` : Az utolsó `UPDATE` , `DELETE` vagy `INSERT` érintett-e legalább egy sort.

- `SQL%ROWCOUNT`: A legutóbbi DML utasítás által érintett sorok száma.

---

6. Kivételkezelés (Exception Handling)

6.1 Beépített kivételek

- **Definíció:** Előre definiált Oracle hibák elkapása a blokk `EXCEPTION` ágában.
- **Legfontosabb kivételek:**
- `NO_DATA_FOUND`: Egy `SELECT ... INTO` lekérdezés 0 sort adott vissza.
- `TOO_MANY_ROWS`: Egy `SELECT ... INTO` 1-nél több sort adott vissza.
- `DUP_VAL_ON_INDEX`: `UNIQUE` vagy `PRIMARY KEY` megszorítás megsértése beszúráskor/módosításkor.
- `ZERO_DIVIDE`: Nullával való osztási kísérlet.
- `VALUE_ERROR`: Méret- vagy típuskonverziós hiba (pl. túl hosszú szöveg).
- `OTHERS`: Minden egyéb nem nevesített kivétel elkapása (`SQLERRM`-mel naplózható).
- **Példa:** `sql EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Nincs adat!'); WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('Túl sok sor!'); WHEN DUP_VAL_ON_INDEX THEN DBMS_OUTPUT.PUT_LINE('Már létező kulcsérték!'); WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Váratlan hiba: ' || SQLERRM); END;`

6.2 Aggregációs csapda (Aggregation Trap)

- **Definíció:** A `COUNT(*)`, `MAX()`, `MIN()`, `SUM()`, `AVG()` aggregáló függvények **mindig pontosan 1 sort adnak vissza** (ha nincs egyezés, akkor 0-t vagy `NULL`-t). Emiatt rájuk a `NO_DATA_FOUND` kivétel **soha nem váltódik ki**!
- **Kezelés:** Ha a feladat kifejezetten kéri a hibakezelést, az `EXCEPTION` ágat formálisan meg kell írni, vagy `COUNT` / `IF v_ertek IS NULL` logikával kell lekezelni.

---

7. Konzolos Kommunikáció (`DBMS_OUTPUT`)

7.1 Metódusok és Karakterek

- **Definíció:** A PL/SQL szerveroldali kimeneti puffere.
- `DBMS_OUTPUT.PUT_LINE(str)`: Kiírja a szöveget és sortörést szúr be.
- `DBMS_OUTPUT.PUT(str)`: Kiírja a szöveget sortörés nélkül.
- `DBMS_OUTPUT.NEW_LINE`: Új sort kezd.
- `chr(10)`: Line feed (LF) karakter; szövegek közé fűzhető sortörésként (`|| chr(10) ||`).
- **Példa:** `sql DBMS_OUTPUT.PUT_LINE('Név: ' || v_nev || chr(10) || 'Ár: ' || v_ar);`

---

8. Tárolt Eljárások és Függvények

8.1 Tárolt Eljárás (`CREATE PROCEDURE`)

- **Definíció:** Önállóan tárolt alprogram, amely műveleteket végez. Nincs közvetlen `RETURN` visszatérési típusa.
- **Paraméter módok:**
- `IN` : Csak olvasható bemeneti paraméter (alapértelmezett).
- `OUT` : Csak írható kimeneti paraméter (eredmény visszaadására hívónak).
- `IN OUT` : Olvasható és írható paraméter.
- **Példa:** `sql CREATE OR REPLACE PROCEDURE modosit_keszlet( p_bt_id IN NUMBER, p_mennyiseg IN NUMBER, p_uj_ertek OUT NUMBER ) IS BEGIN UPDATE bolt_termek SET keszlet = keszlet + p_mennyiseg WHERE id = p_bt_id RETURNING keszlet INTO p_uj_ertek; END modosit_keszlet; /`

8.2 Nevesített paraméterátadás (Named Notation)

- **Definíció:** Az eljárások paramétereinek nevesített átadása `=>` operátorral, tetszőleges sorrendben.
- **Példa:** `sql modosit_keszlet(p_mennyiseg => 10, p_bt_id => 10000, p_uj_ertek => v_keszlet);`

8.3 Lokális (beágyazott) eljárás

- **Definíció:** Egy eljárás vagy függvény deklarációs részében definiált belső segédeljárás, amely csak a szülő alprogramon belül hívható meg.
- **Példa:** `sql CREATE OR REPLACE PROCEDURE print_riport IS PROCEDURE vonal IS BEGIN DBMS_OUTPUT.PUT_LINE('----------------------------'); END; BEGIN vonal; DBMS_OUTPUT.PUT_LINE('Jelentés fejléc'); vonal; END; /`

8.4 Tárolt Függvény (`CREATE FUNCTION`)

- **Definíció:** Olyan alprogram, amely bemenő paraméterek alapján számítást végez, és kötelezően rendelkezik visszatérési értékkel (`RETURN típus`). SQL `SELECT` -ben is használható, ha nem hajt végre DML-t.
- **Példa:** `sql CREATE OR REPLACE FUNCTION get_keszlet_ertek(p_bt_id IN NUMBER) RETURN NUMBER IS v_ertek NUMBER; BEGIN SELECT egysegar * keszlet INTO v_ertek FROM bolt_termek WHERE id = p_bt_id; RETURN v_ertek; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN -1; WHEN TOO_MANY_ROWS THEN RETURN -2; END get_keszlet_ertek; /`

---

9. Csomagok (Packages)

9.1 Fejléc (Specification)

- **Definíció:** A csomag publikus felülete. Tartalmazza a kívülről elérhető függvények, eljárások szignatúráit és globális változóit.

- **Példa:** `sql CREATE OR REPLACE PACKAGE pkg_raktar AS gv_alap_keszlet NUMBER; FUNCTION get_keszlet_ertek(p_bt_id IN NUMBER) RETURN NUMBER; PROCEDURE ar_kalkulacio(p_bt_id IN NUMBER); END pkg_raktar; /`

9.2 Törzs (Body)

- **Definíció:** A csomag megvalósítása. Itt szerepel az összes fejlécben deklarált alprogram teljes kódja, valamint a csomagon belüli privát változók és privát alprogramok.

9.3 Csomagszintű állapot és Inicializációs blokk

- **Definíció:**
- A csomagszintű változók munkamenet-szintűek (`session-level`): a kapcsolat bezárásáig megőrzik az értéküket.
- Az inicializációs blokk a csomagtörzs végén lévő `BEGIN ... END pkg;` szakasz, amely pontosan **egyszer** fut le automatikusan, amikor a munkamenet először hivatkozik a csomagra.

- **Példa:** ```sql CREATE OR REPLACE PACKAGE BODY pkg_raktar AS gv_privat_szamlalo NUMBER := 0; -- privát változó

  FUNCTION get_keszlet_ertek(p_bt_id IN NUMBER) RETURN NUMBER IS ... END; PROCEDURE ar_kalkulacio(p_bt_id IN NUMBER) IS ... END;

BEGIN -- Inicializációs blokk: gv_alap_keszlet := 100; END pkg_raktar; / ```

---

10. Objektum Típusok (Object Types)

10.1 `CREATE TYPE ... AS OBJECT`

- **Definíció:** Kompozit felhasználói típus definiálása attribútumokkal.
- **Példa:** `sql CREATE OR REPLACE TYPE ty_termek_type AS OBJECT ( termek_id NUMBER, megnevezes VARCHAR2(250), CONSTRUCTOR FUNCTION ty_termek_type RETURN SELF AS RESULT ); /`

10.2 Felülírt és Paraméter nélküli konstruktor

- **Definíció:** Lehetővé teszi az objektum inicializálását argumentumok megadása nélkül (`NULL` attribútumokkal).
- **Példa:** `sql CREATE OR REPLACE TYPE BODY ty_termek_type AS CONSTRUCTOR FUNCTION ty_termek_type RETURN SELF AS RESULT IS BEGIN SELF.termek_id := NULL; SELF.megnevezes := NULL; RETURN; END; END; /`

10.3 Konstruktor túlterhelés (Overloading)

- **Definíció:** Egy típushoz több különböző paraméterezésű konstruktor is készíthető (pl. csak ID-t váró, vagy minden mezőt inicializáló).

---

11. Kollekciók (Collections)

11.1 Nested Table (Beágyazott tábla)

- **Definíció:** Olyan méretezhető egydimenziós tömb, amely sémaszinten (`CREATE TYPE ... IS TABLE OF`) is létrehozható, így SQL lekérdezésekben és táblaoszlopként is szerepelhet.
- **Példa:** `sql CREATE OR REPLACE TYPE ty_termek_list IS TABLE OF ty_termek_type; /`

11.2 Asszociatív tömb (Associative Array / `INDEX BY`)

- **Definíció:** Kulcs-érték párokat tároló hash-tábla. Csak PL/SQL blokkokban használható. A kulcs lehet `PLS_INTEGER` vagy `VARCHAR2` .
- **Iterálás:** Mivel a kulcsok ritkák (sparse) lehetnek, iterálni mindig `.FIRST` és `.NEXT()` metódusokkal kell!

- **Példa:** ```sql DECLARE TYPE t_termek_map IS TABLE OF VARCHAR2(250) INDEX BY PLS_INTEGER; v_map t_termek_map; v_idx PLS_INTEGER; BEGIN v_map(10) := 'Tej'; v_map(20) := 'Kenyér';

  v_idx := v_map.FIRST; WHILE v_idx IS NOT NULL LOOP DBMS_OUTPUT.PUT_LINE(v_idx || ' -> ' || v_map(v_idx)); v_idx := v_map.NEXT(v_idx); END LOOP; END; / ```

11.3 Kollekció metódusok

Metódus	Leírás
`.COUNT`	Elemek aktuális száma
`.EXTEND`	Új üres elem hozzáadása a végéhez (Nested Table/Varray)
`.EXISTS(i)`	Igaz, ha létezik elem az adott `i` indexen
`.FIRST / .LAST`	Első és utolsó index lekérdezése
`.PRIOR(i) / .NEXT(i)`	Előző és következő érvényes index lekérdezése
`.DELETE / .DELETE(i)`	Teljes kollekció vagy adott indexű elem törlése

11.4 `MULTISET UNION ALL`

- **Definíció:** Két Nested Table kollekció tartalmi összefűzésére szolgáló beépített operátor.
- **Példa:** `sql v_eredmeny := v_lista1 MULTISET UNION ALL v_lista2;`

---

12. Tömeges Műveletek (`BULK COLLECT`)

12.1 `BULK COLLECT INTO`

- **Definíció:** Kontextusváltás-csökkentő mechanizmus. A lekérdezés teljes eredményhalmazát egyetlen lépésben betölti a memóriában lévő kollekcióba (Nested Table vagy asszociatív tömb).
- **Példa:** `sql DECLARE TYPE t_nevek IS TABLE OF termek.megnevezes%TYPE; v_nevek t_nevek; BEGIN SELECT megnevezes BULK COLLECT INTO v_nevek FROM termek ORDER BY megnevezes; FOR i IN 1..v_nevek.COUNT LOOP DBMS_OUTPUT.PUT_LINE(v_nevek(i)); END LOOP; END; /`

12.2 `FETCH ... BULK COLLECT INTO ... LIMIT`

- **Definíció:** Nagy adatmennyiségek szakaszos (kötegelt) feldolgozása a memóriatúlcsordulás megelőzésére.
- **Példa:** `sql OPEN cur; LOOP FETCH cur BULK COLLECT INTO v_batch LIMIT 100; EXIT WHEN v_batch.COUNT = 0; -- feldolgozás... END LOOP; CLOSE cur;`

---

13. Szövegműveletek és Eredményhalmaz-összefűzés

13.1 `LISTAGG`

- **Definíció:** Eredménysorok oszlopértékeit összefűző aggregáló függvény `WITHIN GROUP (ORDER BY ...)` klauzulával.
- **Példa:** `sql SELECT LISTAGG(t.megnevezes, ', ') WITHIN GROUP (ORDER BY t.megnevezes) INTO v_termek_lista FROM bolt_termek bt JOIN termek t ON t.id = bt.termek_id WHERE bt.bolt_id = 12300;`

---

14. Szekvenciák (Sequences)

14.1 `CREATE SEQUENCE`, `.NEXTVAL`, `.CURRVAL`

- **Definíció:** Adatbázisszintű sorszámgenerátor kulcsértékek (`ID`) előállítására.
- **Példa:** ```sql CREATE SEQUENCE seq_bolt_termek_id START WITH 20000 INCREMENT BY 1 NOCACHE;

-- Használat INSERT-ben: INSERT INTO bolt_termek(id, bolt_id, termek_id, egysegar, keszlet) VALUES (seq_bolt_termek_id.NEXTVAL, 12300, 9000, 350, 100);

-- Generált érték lekérdezése: v_id := seq_bolt_termek_id.CURRVAL; ```

---

15. Dinamikus SQL (`EXECUTE IMMEDIATE`)

15.1 Futásidejű SQL és DDL futtatása

- **Definíció:** Futásidőben összefűzött SQL vagy DDL szövegek végrehajtása.
- **Szabály:** DML esetén bind változókat használunk (`:1`, `USING`), DDL esetén (pl. `CREATE TABLE`) nem lehet bind változót használni.
- **Példa:** ```sql -- DML lekérdezés bind változóval: EXECUTE IMMEDIATE 'SELECT megnevezes FROM termek WHERE id = :1' INTO v_nev USING 9000;

-- DDL futtatása dinamikusan: EXECUTE IMMEDIATE 'CREATE TABLE temp_adatok (id NUMBER)'; ```

---

16. Gyakori Beépített Függvények PL/SQL-ben

16.1 String és NULL kezelés

- `SUBSTR(s, start, len)`: Részsztring kinyerése.
- `INSTR(s, ker)`: Karakterlánc pozíciójának megkeresése.
- `LENGTH(s)`: Hosszúság lekérdezése.
- `TRIM(s)`: Vezető és záró szóközök levágása.
- `NVL(kif, potas)`: Ha `kif` NULL, a pótló értéket adja vissza.
- `NVL2(kif, ha_nem_null, ha_null)`: Háromparaméteres elágazás NULL-ra.
- `COALESCE(a, b, c)`: Az első nem-NULL értéket adja vissza.

16.2 Konverziók és Dátumkezelés

- `TO_CHAR(d, 'YYYY.MM.DD HH24:MI:SS')`: Dátum formázott szöveggé alakítása.
- `TO_DATE('2024.01.15', 'YYYY.MM.DD')`: Szöveg dátummá konvertálása.
- `TO_NUMBER('1234')`: Szöveg számmá konvertálása.
- `ROUND(szam, tizedes)`: Kerekítés adott tizedesjegyre.
- `MOD(a, b)`: Maradékos osztás.
- `SYSDATE`: Aktuális szerveroldali dátum és időpont.

---

17. Tranzakciókezelés és Jogosultságok

17.1 Tranzakciók (`COMMIT`, `ROLLBACK`, `SAVEPOINT`)

- **Definíció:**
- `COMMIT`: A tranzakció során végzett DML műveletek véglegesítése.
- `ROLLBACK`: A tranzakció visszavonása az utolsó commitig.
- `SAVEPOINT pont_nev`: Részleges mentési pont létrehozása (`ROLLBACK TO pont_nev`).
- **Fontos:** Bármely DDL utasítás (pl. `CREATE`, `ALTER`, `DROP`, `GRANT`) **automatikus implicit COMMIT-ot** hajt végre előtte és utána is!

17.2 Jogosultságok (`GRANT`)

- `GRANT CREATE TYPE TO felhasználó` : Szükséges saját objektumtípusok definiálásához.
- `GRANT CREATE PROCEDURE TO felhasználó` : Szükséges tárolt eljárások, függvények és csomagok létrehozásához.

---

18. Gyors Áttekintő Táblázat a ZH-ra

Téma	Szintaxis Kulcsszavak	Tipikus Feladat / Felhasználás
Object Type	`CREATE TYPE ty_x AS OBJECT (...)`	Rekord-struktúra definiálása
Constructor	`CONSTRUCTOR FUNCTION ty_x RETURN SELF AS RESULT`	Paraméter nélküli/üres objektum példányosítása
Függvény	`CREATE FUNCTION fn RETURN ty_x IS ...`	Lekérdezés, adatok visszaadása típussal
Összefűzés	`LISTAGG(nev, ',') WITHIN GROUP (ORDER BY nev)`	Eredménysorok rendezett vesszős összefűzése
Kivételkezelés	`WHEN NO_DATA_FOUND THEN ... WHEN TOO_MANY_ROWS`	Nem létező rekord vagy több találat lekezelése
Csomag Fejléc	`CREATE PACKAGE pkg AS gv_var NUMBER; ... END;`	Publikus interfész és csomagváltozó
Csomag Törzs	`CREATE PACKAGE BODY pkg AS ... BEGIN init; END;`	Implementáció és induló inicializálás
Eljárás	`CREATE PROCEDURE prc(p_id NUMBER) IS ...`	Ármódosítás, készletkezelés, kiíratás
Kurzor FOR LOOP	`FOR rec IN cur LOOP ... END LOOP;`	Több soros eredmény bejárása és kiírása
Asszociatív Tömb	`TABLE OF ... INDEX BY PLS_INTEGER;`	Memóriabeli kulcs-érték tároló
BULK COLLECT	`SELECT ... BULK COLLECT INTO kollekció`	Tömeges sorbeolvasás memóriába
DML Modosítás	`UPDATE tabla SET oszlop = ... WHERE ...;`	Rekordok módosítása (pl. áremelés)
Kimenet	`DBMS_OUTPUT.PUT_LINE('szöveg' || var);`	Eredmények kiíratása konzolra